

ALL DEVOPS LABS

Podman basics

The lab command starts the `basics-podman-secret` container, which contains the `/etc/secret-file` file.

Copy the `/etc/secret-file` file from the container to the `/home/student/D0188/labs/basics-podman/solution` file.

1. Use the `podman ps` command to verify that the `basics-podman-secret` container is running.

```
[student@workstation ~]$ podman ps --format='{{.Names}}'
basics-podman-secret
```

2. Use the `ls` command inside of the container to verify that the container contains the `/etc/secret-file` file.

```
[student@workstation ~]$ podman exec basics-podman-secret ls /etc/secret-file
/etc/secret-file
```

3. Change to the `/home/student/D0188/labs/basics-podman` directory, for example:

```
[student@workstation ~]$ cd ~/D0188/labs/basics-podman
[student@workstation basics-podman]$ ls
index.html
```

4. Copy the `/etc/secret-file` file from the container as the `solution` file in the `/home/student/D0188/labs/basics-podman` directory.

```
[student@workstation basics-podman]$ podman cp \
  basics-podman-secret:/etc/secret-file solution
[student@workstation basics-podman]$ ls
index.html  solution
```

Start a new container with the following parameters:

- Name: basics-podman-server
- Image: registry.ocp4.example.com:8443/ubi9/httpd-24
- Ports: Route traffic from port 8080 on your machine to port 8080 inside of the container
- Network: lab-net

You can start the container in the detached mode for greater convenience.

1. Create the lab-net Podman network.

```
[student@workstation basics-podman]$ podman network create lab-net
lab-net
```

The next objective of the lab script is passing.

2. Execute the podman run command to start the container.

```
[student@workstation basics-podman]$ podman run -d --name basics-podman-server \
  --net lab-net -p 8080:8080 registry.ocp4.example.com:8443/ubi9/httpd-24
8b747...3616
```

Copy the /home/student/D0188/labs/basics-podman/index.html file to /var/www/html/ in the basics-podman-server container.

1. Verify that you are in the correct directory.

```
[student@workstation basics-podman]$ ls
index.html  solution
```

2. Copy the index.html file in to the container.

```
[student@workstation basics-podman]$ podman cp index.html \
  basics-podman-server:/var/www/html/In a web browser, go to http://localhost:8080 and
  verify that you see the Hello from Podman Basics lab text.
```

Start a new container with the following parameters:

- Name: basics-podman-client
- Image: registry.ocp4.example.com:8443/ubi9/httpd-24
- Network: lab-net

You can start the container in the detached mode for greater convenience. Execute the `podman run` command to start the container.

```
[student@workstation basics-podman]$ podman run -d --name basics-podman-client \  
--net lab-net registry.ocp4.example.com:8443/ubi9/httpd-24  
8b747...3616
```

Container Images

Build a container image that uses the `/home/student/D0188/labs/images-lab/Containerfile` file.

Call the resulting image `images-lab` and push the image into the `registry.ocp4.example.com:8443` registry in the developer user repository.

Use the developer user with the developer password to authenticate with the `registry.ocp4.example.com:8443` registry.

Authenticate Podman with the `registry.ocp4.example.com:8443` registry.

```
[student@workstation ~]$ podman login -u developer -p developer \
    registry.ocp4.example.com:8443
Login Succeeded!
```

In a terminal, change to the `/home/student/D0188/labs/images-lab` directory.

```
[student@workstation ~]$ cd ~/D0188/labs/images-lab
no output expected
```

Build the Containerfile with the `registry.ocp4.example.com:8443/developer/images-lab` container image name.

```
[student@workstation images-lab]$ podman build --file Containerfile \
    --tag registry.ocp4.example.com:8443/developer/images-lab...output omitted...
Successfully tagged registry.ocp4.example.com:8443/developer/images-lab:latest
8d14...dd5a
```

Push the image to the `registry.ocp4.example.com:8443` registry.

```
[student@workstation images-lab]$ podman push registry.ocp4.example.com:8443/developer/images-lab
...output omitted...
Writing manifest to image destination
```

Add the `grue` tag to the `images-lab` container image, and push it to the `registry.ocp4.example.com:8443` registry.

Add the `grue` tag to the image.

```
[student@workstation images-lab]$ podman tag \
```

```
registry.ocp4.example.com:8443/developer/images-lab \  
  
registry.ocp4.example.com:8443/developer/images-lab:grue  
no output expected
```

Push the new image tag.

```
[student@workstation images-lab]$ podman push \  
registry.ocp4.example.com:8443/developer/images-lab:grue  
...output omitted...  
Writing manifest to image destination
```

Create a container by using the `images-lab:grue` image. Use the `images-lab` container name, and bind the `8080` container port to the `8080` host port. Start the container in the background.

Optionally, use the `curl` command to verify that the HTTP server is running.

Start the container.

```
[student@workstation images-lab]$ podman run -d --name images-lab \  
-p 8080:8080 images-lab:grue  
2422...e7b1
```

Connect to the HTTP server by using `curl`. Note that this step is not required to make the lab script pass.

```
[student@workstation images-lab]$ curl localhost:8080
```

Custom Container Images

Go to the `/home/student/D0188/labs/custom-lab` directory, which contains the application that converts a text into a QR code image. Then, run the app on the host machine by using the `npm install` and `npm start` commands. Verify that the application fails gracefully because an environment variable is missing.

Go to the `~/D0188/labs/custom-lab` directory.

```
[student@workstation ~]$ cd ~/D0188/labs/custom-lab  
no output expected
```

Install the application dependencies.

```
[student@workstation custom-lab]$ npm install  
added 201 packages, and audited 202 packages in 1s  
...output omitted...
```

Start the application. The application exits because the HTTP port is not set.

```
[student@workstation custom-lab]$ npm start  
> custom-images-lab@1.0.0 start  
> node index.js  
HTTP PORT not found. Set the env variable to proceed.
```

In the build stage of the Containerfile, generate the TLS certificates by using the `./gen_certificates.sh` command.

The `./gen_certificates.sh` command is included in the provided container.

Use the `RUN` instruction to generate the TLS certificates.

```
FROM registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \  
as certs  
  
RUN ./gen_certificates.sh  
  
FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1  
USER root  
RUN groupadd -r student && useradd -r -m -g student student && \
```

```
npm config set cache /tmp/.npm --global
```

```
COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
```

```
COPY --chown=student:student . /app/
```

In the final stage of the Containerfile, set the following environment variables:

- TLS_PORT=8443 (the port for TLS traffic)
- HTTP_PORT=8080 (the port for HTTP traffic)
- CERTS_PATH=/etc/pki/tls/private/certs (the path that contains the TLS certificates)

Build the container image with the name localhost/podman-qr-app.

Use the ENV instruction to add the environment variables to the Containerfile.

```
FROM \
    registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \
    as certs

RUN ./gen_certificates.sh

FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1
USER root
RUN groupadd -r student && useradd -r -m -g student student && \
    npm config set cache /tmp/.npm --global

COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
COPY --chown=student:student . /app/

ENV TLS_PORT=8443 \
    HTTP_PORT=8080 \
    CERTS_PATH="/etc/pki/tls/private/certs"
```

Build the container image.

```
[student@workstation custom-lab]$ podman build -t localhost/podman-qr-app .
...output omitted...
Successfully tagged localhost/podman-qr-app:latest
201...cc8
```

In the final stage of the Containerfile, set the working directory of the application to the /app path.

Then, build the container image with the name localhost/podman-qr-app.

Use the WORKDIR instruction to define the working directory.

```
FROM \
    registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \
    as certs

RUN ./gen_certificates.sh

FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1
USER root
RUN groupadd -r student && useradd -r -m -g student student && \
    npm config set cache /tmp/.npm --global

COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
COPY --chown=student:student . /app/

ENV TLS_PORT=8443 \
    HTTP_PORT=8080 \
    CERTS_PATH="/etc/pki/tls/private/certs"
WORKDIR /app
```

Build the container image.

```
[student@workstation custom-lab]$ podman build -t localhost/podman-qr-app .
...output omitted...
Successfully tagged localhost/podman-qr-app:latest
201...cc8
```

In the final stage of the Containerfile, set the student user as the user that runs the application. The student user exists in the Containerfile. Then, build the container image with the name localhost/podman-qr-app.

Use the USER instruction.

```
FROM \
```

```

registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \
as certs
RUN ./gen_certificates.sh

FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1
USER root
RUN groupadd -r student && useradd -r -m -g student student && \
    npm config set cache /tmp/.npm --global

COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
COPY --chown=student:student . /app/

ENV TLS_PORT=8443 \
    HTTP_PORT=8080 \
    CERTS_PATH="/etc/pki/tls/private/certs"

WORKDIR /app
USER student

```

Build the container image.

```

[student@workstation custom-lab]$ podman build -t localhost/podman-qr-app .
...output omitted...
Successfully tagged localhost/podman-qr-app:latest
201a...ecc8

```

In the final stage of the Containerfile, run the `npm install --omit=dev` command to install the production dependencies of the Node.js application.

Then, build the container image with the name `localhost/podman-qr-app`.

Use the `RUN` instruction to execute the command.

```

FROM \
    registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \
as certs

RUN ./gen_certificates.sh

FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1

```

```

USER root
RUN groupadd -r student && useradd -r -m -g student student && \
    npm config set cache /tmp/.npm --global
COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
COPY --chown=student:student . /app/

ENV TLS_PORT=8443 \
    HTTP_PORT=8080 \
    CERTS_PATH="/etc/pki/tls/private/certs"

WORKDIR /app

USER student
RUN npm install --omit=dev

```

Build the container image.

```

[student@workstation custom-lab]$ podman build -t localhost/podman-qr-app .
...output omitted...
Successfully tagged localhost/podman-qr-app:latest
201...cc8

```

In the final stage of the Containerfile, make `npm` start the default command for this image. Additional runtime arguments should not override the default command.

Then, build the container image with the name `localhost/podman-qr-app`.

Use the `ENTRYPOINT` instruction to execute the command when the container is started.

```

FROM \
    registry.ocp4.example.com:8443/redhattraining/podman-certificate-generator \
    as certs
RUN ./gen_certificates.sh

FROM registry.ocp4.example.com:8443/ubi9/nodejs-22:1
USER root
RUN groupadd -r student && useradd -r -m -g student student && \
    npm config set cache /tmp/.npm --global

```

```
COPY --from=certs --chown=student:student /app/*.pem /etc/pki/tls/private/certs/
COPY --chown=student:student . /app/

ENV TLS_PORT=8443 \
    HTTP_PORT=8080 \
    CERTS_PATH="/etc/pki/tls/private/certs"

WORKDIR /app

USER student

RUN npm install --omit=dev

ENTRYPOINT npm start
```

Build the container image.

```
[student@workstation custom-lab]$ podman build -t localhost/podman-qr-app .
...output omitted...
Successfully tagged localhost/podman-qr-app:latest
201...cc8
```

Start the `podman-qr-app` container. Call the container `custom-lab` and publish the container port `8080` and `8443` to the same ports on the host.

Use the `podman run` command to start the application and bind the corresponding ports.

```
[student@workstation custom-lab]$ podman run --name=custom-lab \
    -p 8080:8080 -p 8443:8443 podman-qr-app
...output omitted...
TLS Server running on port 8443
Server running on port 8080
```

Optionally, test the application by navigating to `http://localhost:8080` in a web browser.

Persisting data

Create a named volume with the following parameters:

- The volume is called `postgres-vol`.
- The volume contains the contents of the `/home/student/D0188/labs/persisting-lab/postgres-vol.tar.gz` file.

Create the volume.

```
[student@workstation ~]$ podman volume create postgres-vol  
postgres-vol
```

Import the `/home/student/D0188/labs/persisting-lab/postgres-vol.tar.gz` file to the volume:

```
[student@workstation ~]$ podman volume import postgres-vol \  
~/D0188/labs/persisting-lab/postgres-vol.tar.gz  
..no output expected...
```

Start the application database container with the following parameters:

- Call the container `persisting-db`.
- Start the container in the background.
- Connect the container to the `persisting-net` network.
- Use the following environment variables:
 - `POSTGRESQL_PASSWORD=pass`
 - `POSTGRESQL_USER=user`
 - `POSTGRESQL_DATABASE=db`
- Mount the `postgres-vol` volume to the `/var/lib/pgsql/data` directory.
- Use the `registry.ocp4.example.com:8443/rhel9/postgresql-13:1` image.

Create the `persisting-net` network.

```
[student@workstation ~]$ podman network create persisting-net  
persisting-net
```

Start the database container.

```
[student@workstation ~]$ podman run --name persisting-db -d \  
--net persisting-net -e POSTGRESQL_USER=user -e POSTGRESQL_PASSWORD=pass \  
-e POSTGRESQL_DATABASE=db \  
registry.ocp4.example.com:8443/rhel9/postgresql-13:1
```

```
--mount='type=volume,src=postgres-vol,dst=/var/lib/pgsql/data' \  
registry.ocp4.example.com:8443/rhel9/postgresql-13:1  
c97f...4a29
```

Start the front end with the following parameters:

- Call the container `persisting-frontend`.
- Start the container in the background.
- Connect the container to the `persisting-net` network.
- Expose the port `3000` on the machine to route requests to port `8080` inside the container.
- Use the `registry.ocp4.example.com:8443/redhattraining/podman-urlshortener-frontend` image.

```
[student@workstation ~]$ podman run --name persisting-frontend -d \  
--net persisting-net -p 3000:8080 \  
registry.ocp4.example.com:8443/redhattraining/podman-urlshortener-frontend  
b10e...940f
```

Test the application.

In a web browser, verify the functionality of the application at `http://localhost:3000`.

In a web browser, test the database data import by navigating to `http://localhost:8080/api/shorturl/a9yi4rc15uuzunv`.

The `a9yi4rc15uuzunv` short URL is a part of the database data that you imported in a previous step into the `postgres-vol` volume.

Troubleshooting

Verify that the following containers are running with the following configuration:

quotes-api-v1 and quotes-api-v2

Container image: registry.ocp4.example.com:8443/redhattraining/wiremock

quotes-ui

Port mapping: 3000:8080

Container image: registry.ocp4.example.com:8443/redhattraining/quotes-ui-versioning:1.0

Environment variable: QUOTES_API_VERSION=v2

If any of the containers are missing, then read the container logs to determine what is preventing the container from starting. Then, fix the issue and start the container.

Use the following troubleshooting guide to identify and fix the problems:

Problem: unable to retrieve auth token: invalid username/password: unauthorized

Solution: You must log into the image registry.

Problem: requires at least 1 arg(s), only received 0

Solution: Correct your Podman command.

Problem: nginx: [emerg] host not found in upstream

Solution: Nginx reverse proxy cannot resolve hostname to IP address, probably due to a missing Podman network.

Run the `podman ps` command with the `-a` option to show stopped containers.

```
[student@workstation ~]$ podman ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b72652504844	...wiremock:latest		28 sec...	Up...		quotes-api-v1
a3a2f6809446	...wiremock:latest		27 sec...	Up...		quotes-api-v2
867f6f559629	...quotes-ui-ve...	/bin...	21 sec...	Exited...	0...	quotes-ui

The quotes-ui container failed to start.

Read the quotes-ui logs with the `podman logs` command to look for container start-up errors.

```
[student@workstation ~]$ podman logs quotes-ui
nginx: [emerg] host not found in upstream "quotes-api-v1" in /etc/nginx/nginx.conf:45
```

The container fails to start because the NGINX proxy in the quotes-ui container cannot resolve the quotes-api-v1 hostname to an IP address.

Use the `podman inspect` command to determine the Podman networks that the containers use to communicate.

Inspect the quotes-api-v1 networks.

```
[student@workstation ~]$ podman inspect quotes-api-v1 --format='{{.NetworkSettings.Networks}}'
map[troubleshooting-lab:0xc000a825a0]
```

Inspect the quotes-api-v2 networks.

```
[student@workstation ~]$ podman inspect quotes-api-v2 --format='{{.NetworkSettings.Networks}}'
map[troubleshooting-lab:0xc000a825a0]
```

Repeat the command for the quotes-ui container.

```
[student@workstation ~]$ podman inspect quotes-ui --format='{{.NetworkSettings.Networks}}'
map[pasta:0xc0006cd200]The quotes-ui container is missing the troubleshooting-lab network and thus it cannot resolve the quotes-api-v1 and quotes-api-v2 hostnames.
```

Remove the current quotes-ui container.

```
[student@workstation ~]$ podman rm quotes-ui
quotes-ui
```

Recreate the quotes-ui container and attach it to the troubleshooting-lab network.

```
[student@workstation ~]$ podman run -d \
  --name quotes-ui -p 3000:8080 \
  -e QUOTES_API_VERSION=v2 \
  --net troubleshooting-lab \
  registry.ocp4.example.com:8443/redhattraining/quotes-ui-versioning:1.0
d838...7432
```

Verify that the quotes-ui container starts.

```
[student@workstation ~]$ podman ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b72652504844	...wiremock:latest		28 sec...	Up...		quotes-api-v1
a3a2f6809446	...wiremock:latest		27 sec...	Up...		quotes-api-v2
6e8581a9d73b	...quotes-ui-versioning:1.0		1 sec...	Up...		quotes-ui

Ensure that the NGINX reverse proxy can route requests to both versions of the quotes API.

The NGINX reverse proxy in the quotes-ui container should route requests as indicated in the following table:

Table 6.1. NGINX mappings

From	To
http://localhost:8080/api/v1/quotes	http://quotes-api-v1:8080/api/v1/quotes
http://localhost:8080/api/v2/quotes	http://quotes-api-v2:8081/api/v2/quotes

The quotes-ui logs provide some network information that you might need.

Perform any configuration modifications in the quotes-ui container.

Use a bind mount to replace the /etc/nginx/nginx.conf file in the quotes-ui container with a modified /home/student/D0188/labs/troubleshooting-lab/nginx.conf file, if necessary.

Use the podman exec command to run the curl command from the quotes-ui container to test the v1 and v2 mappings. Use the localhost hostname to test the NGINX mappings. Use the silent option (-s) with the curl command to print the output only.

Test that the v1 mapping works.

```
[student@workstation ~]$ podman exec quotes-ui \
  curl -s http://localhost:8080/api/v1/quotes
```

```
[
  {
    "id": 1,
    "quote": "I live my daydreams in music.",
    "author": "Albert Einstein"
  },
```

```
...output omitted...
```

Test that the v2 mapping does not work.

```
[student@workstation ~]$ podman exec quotes-ui \
  curl -s http://localhost:8080/api/v2/quotes
<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.20.1</center>
</body>
</html>
```

This error means that NGINX was able to map the request but did not get a response from the quotes-api-v2 service.

Investigate if the quotes-api-v2 service is using a different port. Run the `podman logs` command on the quotes-api-v2 container to find the port that the service is using.

```
[student@workstation ~]$ podman logs quotes-api-v2
...output omitted...
port: 8081
...output omitted...
```

The quotes-api-v2 service is using the 8081 port.

Read the quotes-ui container NGINX configuration file at the `/etc/nginx/nginx.conf` path.

```
[student@workstation ~]$ podman exec quotes-ui \
  cat /etc/nginx/nginx.conf
...output omitted...
    location /api/v2 {
        rewrite /api/v2/(.*) /$1 break;
        proxy_pass http://quotes-api-v2:8080;
    }
...output omitted...
```

The mapping for v2 is redirecting requests to the `http://quotes-api-v2:8080` address, which points to the wrong port.

Read the updated copy of the NGINX configuration at `/home/student/D0188/labs/troubleshooting-lab/nginx.conf`.

```
[student@workstation ~]$ cat ~/D0188/labs/troubleshooting-lab/nginx.conf
...output omitted...
    location /api/v2 {
        rewrite /api/v2/(.*) /$1 break;
        proxy_pass http://quotes-api-v2:8081;
    }
...output omitted...
```

This version points to the right 8081 port.

Remove the `quotes-ui` container by running `podman rm` using the `-f` option.

```
[student@workstation ~]$ podman rm -f quotes-ui
quotes-ui
```

Recreate the `quotes-ui` container. Override the container's `/etc/nginx/nginx.conf` file with the `/home/student/D0188/labs/troubleshooting-lab/nginx.conf` file by using a bind mount with the `:z` option.

```
[student@workstation ~]$ podman run -d \
    --name quotes-ui \
    -p 3000:8080 \
    -e QUOTES_API_VERSION=v2 \
    --net troubleshooting-lab \
    -v ~/D0188/labs/troubleshooting-lab/nginx.conf:/etc/nginx/nginx.conf:Z \
    registry.ocp4.example.com:8443/redhattraining/quotes-ui-versioning:1.0
a0d1...a73f
```

Confirm that the application is accessible by using a web browser to go to `http://localhost:3000`.

Compose

Change to the `/home/student/D0188/labs/compose-lab` directory. This directory contains the compose file called `compose.yaml`, which you must complete. Then, start the containers in the background.

Container Name	Image
quotes-provider	registry.ocp4.example.com:8443/redhattraining/wiremock
quotes-api	registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose
quotes-ui	registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui

NOTE

The WireMock container image provides the ability to create a fake API service. This exercise uses this image to run the quotes-provider API. In a real-world scenario, your API image is built from your application source code.

Change to the `/home/student/D0188/labs/compose-lab` directory.

```
[student@workstation compose-lab]$ cd ~/D0188/labs/compose-lab  
no output expected
```

Update the `compose.yaml` file to look like the following code block:

```
name: compose-lab  
services:  
  wiremock:  
    container_name: "quotes-provider"  
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"  
  quotes-api:  
    container_name: "quotes-api"  
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"  
  quotes-ui:  
    container_name: "quotes-ui"    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
```

From the directory that contains the `compose.yaml` file, run `podman-compose up` to create the containers. Use the `-d` option to run the containers in the background.

```
[student@workstation compose-lab]$ podman-compose up -d
quotes-provider
...output omitted...
quotes-api
...output omitted...
quotes-ui
...output omitted...
```

Configure the `quotes-provider` mock service by providing the Wiremock mappings and responses in the `/home/student/D0188/labs/compose-lab/wiremock/stubs` directory.

Wiremock expects two directories to configure the mock server:

`mappings`: contains files that define the HTTP endpoints.

`__files`: contains files with fixed responses.

To provide the Wiremock configuration, mount the `/home/student/D0188/labs/compose-lab/wiremock/stubs` directory as the `/home/wiremock` directory in the container.

To apply the changes, delete and restart the containers by using the `podman-compose` command.

Use a bind mount to map `/home/student/D0188/labs/compose-lab/wiremock/stubs` to the `/home/wiremock` directory. Provide the `z` option to add the correct SELinux permissions.

```
name: compose-lab

services:
  wiremock:
    container_name: "quotes-provider"
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"
    volumes:
      - ~/D0188/labs/compose-lab/wiremock/stubs:/home/wiremock:Z
  quotes-api:
    container_name: "quotes-api"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"
  quotes-ui:
    container_name: "quotes-ui"
```

```
image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
```

Apply the changes by running the `podman-compose down` command followed by `podman-compose up -d`

Run `podman-compose down` to stop and remove the containers in the `compose.yaml` file.

```
[student@workstation compose-lab]$ podman-compose down
quotes-ui
quotes-provider
quotes-api
quotes-ui
quotes-api
quotes-provider
c08e...78de
compose-lab_default
```

Run `podman-compose up` to re-create the containers with the new changes.

```
[student@workstation compose-lab]$ podman-compose up -d
7c6f...be17
7ddb...f6c3
quotes-provider
b448...4746
quotes-api
01d9...8807
quotes-ui
```

Isolate the `quotes-provider` container from the `quotes-ui` container by creating the following Podman networks in the `compose.yaml` file:

`backend-net`, accessible to the `quotes-api` and `quotes-provider` containers

`frontend-net`, accessible to the `quotes-api` and the `quotes-ui` containers

Note that the `quotes-ui` container must be able to reach the `quotes-api` container for the application to work. For this reason, both containers must be in the `frontend-net` network.

To apply the changes, delete and restart the containers by using the `podman-compose` command.

Use the networks top-level property to create the backend-net and the front-end networks from the `compose.yaml` file. Then use the networks property under every service to connect the containers:

Connect the `quotes-api` and the `quotes-provider` containers by using the `backend-net` network.

Connect the `quotes-api` and the `quotes-ui` containers by using the `frontend-net` network.

```
name: compose-lab

services:
  wiremock:
    container_name: "quotes-provider"
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"
    volumes:
      - ~/D0188/labs/compose-lab/wiremock/stubs:/home/wiremock:Z
    networks:
      - backend-net
  quotes-api:
    container_name: "quotes-api"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"
    networks:
      - backend-net
      - frontend-net
  quotes-ui:
    container_name: "quotes-ui"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
    networks:
      - frontend-net

networks:
  backend-net: {}
  frontend-net: {}
```

Apply the changes by running the `podman-compose down` command followed by the `podman-compose up -d` command.

Run the `podman-compose down` command to stop and remove the containers in the `compose.yaml` file.

```
[student@workstation compose-lab]$ podman-compose down
```

```
...output omitted...
```

Run the `podman-compose up` command to re-create the containers with the new changes.

```
[student@workstation compose-lab]$ podman-compose up -d
...output omitted...
```

The `quotes-api` and `quotes-provider` services share a network, but the `quotes-api` service is missing the hostname configuration of the provider endpoint. Set the `QUOTES_SERVICE` environment variable to configure the `quotes-provider` URL. Use the default name resolution, the `quotes-provider` configuration, the `http` protocol, and the port `8080` to configure this value.

To apply the changes, delete and restart the containers by using the `podman-compose` command.

Add the `QUOTES_SERVICE="http://quotes-provider:8080"` environment variable to the `compose.yaml` file.

```
name: compose-lab

services:
  wiremock:
    container_name: "quotes-provider"
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"
    volumes:
      - ~/D0188/labs/compose-lab/wiremock/stubs:/home/wiremock:Z
    networks:
      - backend-net
  quotes-api:
    container_name: "quotes-api"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"
    networks:
      - backend-net
      - frontend-net
    environment:
      QUOTES_SERVICE: "http://quotes-provider:8080"
  quotes-ui:
    container_name: "quotes-ui"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
```

```
networks:
  - frontend-net

networks:
  backend-net: {}
  frontend-net: {}
```

Apply the changes by running the `podman-compose down` command followed by the `podman-compose up -d` command.

Run the `podman-compose down` command to stop and remove the containers in the `compose.yaml` file.

```
[student@workstation compose-lab]$ podman-compose down
...output omitted...
```

Run the `podman-compose up` command to re-create the containers with the new changes.

```
[student@workstation compose-lab]$ podman-compose up -d
...output omitted...
```

Expose the `quotes-ui` service on port `3000` so that the host can access the application on <http://localhost:3000>. The `quotes-ui` is listening on port `8080` in the UI container.

To apply the changes, delete and restart the containers by using the `podman-compose` command.

Use the previous URL to visit the quotes application by using a web browser.

Use the `ports` property in the `quotes-ui` service to expose the port `8080` in the container to port `3000` on the host.

```
name: compose-lab

services:
  wiremock:
    container_name: "quotes-provider"
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"
    volumes:
      - ~/D0188/labs/compose-lab/wiremock/stubs:/home/wiremock:Z
    networks:
```

```

    - backend-net
quotes-api:
  container_name: "quotes-api"
  image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"
  networks:
    - backend-net
    - frontend-net
  environment:
    QUOTES_SERVICE: "http://quotes-provider:8080"
quotes-ui:
  container_name: "quotes-ui"
  image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
  networks:
    - frontend-net
  ports:
    - "3000:8080"

networks:
  backend-net: {}
  frontend-net: {}

```

Apply the changes by running the `podman-compose down` command followed by the `podman-compose up -d` command.

Run the `podman-compose down` command to stop and remove the containers in the `compose.yaml` file.

```

[student@workstation compose-lab]$ podman-compose down
...output omitted...

```

Run the `podman-compose up` command to re-create the containers with the new changes.

```

[student@workstation compose-lab]$ podman-compose up -d
...output omitted...

```

Go to <http://localhost:3000> to verify that the quotes application shows famous quotes in the browser.